

PropertyMax REST API — Full Fetch Audit Report

ApartmentCorp Operations Hub Date: May 23, 2026 **Prepared by:** Manus AI — Operations Hub Audit

Endpoint Audited

All calls target the same endpoint:

```
GET https://propertymax.ai/rest/max/users/{API_KEY}
```

This endpoint returns the full list of managers and their associated KPI data (`task_counts` , `role` , `regions` , `property_name` , `last_login` , etc.).

Full Fetch Inventory

#	Page / Component	Caller Type	Trigger	Auto-Refresh	Timeout	Retry Logic
1	Home Page — Marketing Stats	<code>useMarketingStats()</code> hook (client-side)	On page mount	None	45 seconds	None
2	Home Page — Portfolio KPIs	<code>usePortfolioKPIs()</code> hook (client-side)	On page mount	None	45 seconds	None
3	Home Page — Manager List / Live Property Count	<code>usePropertyMaxManagers()</code> hook (client-side)	On page mount	None	45 seconds	None
4	KPI Dashboard	Direct <code>fetch()</code> in page component (client-side)	On page mount	Every 5 minutes via <code>setInterval</code>	60 seconds	Auto-retry once after 3 seconds on timeout
5	Marketing Reports	Direct <code>fetch()</code> in page component (client-side)	On page mount	None	60 seconds	Auto-retry once after 3 seconds on timeout
6	Properties Page	Direct <code>fetch()</code> in page component (client-side)	On page mount	None	Browser default (~2 min)	None
7	Server-side <code>managers.getKPIs</code>	tRPC procedure — <code>server/routers/managers.ts</code>	Never called (dead code)	N/A	15 seconds	None

Key Findings

Finding 1 — Home Page Makes 3 Separate API Calls Per Load

The Home page mounts three independent hooks — `useMarketingStats`, `usePortfolioKPIs`, and `usePropertyMaxManagers` — each of which fires its own `fetch()` request to the same PropertyMax endpoint simultaneously. All three calls retrieve the same full user dataset and parse different fields from it. This results in **3 simultaneous API calls** to PropertyMax every time any user opens the Home page.

Finding 2 — KPI Dashboard Auto-Refreshes Every 5 Minutes

The KPI Dashboard uses a `setInterval` to re-fetch the PropertyMax API every 5 minutes while the page is open. This is in addition to the initial load call. A user who leaves the KPI Dashboard tab open for one hour will generate approximately **12 additional API calls** during that session.

Finding 3 — Server-Side Proxy Exists But Is Never Used

A server-side tRPC procedure (`managers.getKPIs` in `server/routers/managers.ts`) was built to proxy PropertyMax API calls through the backend with a 15-second timeout. However, **no page or component currently calls this procedure**. The KPI Dashboard, Marketing Reports, and Properties pages all bypass it and call PropertyMax directly from the user's browser. This means:

- There is no server-side caching layer
- API credentials are exposed in client-side JavaScript
- Every connected user generates independent API calls

Finding 4 — No Caching Layer Exists

Every call goes directly to PropertyMax with no intermediate cache. If 5 users have the Operations Hub open simultaneously and all navigate to the Home page at the same time, PropertyMax could receive up to **15 simultaneous requests** (3 per user). There is no deduplication, no stale-while-revalidate strategy, and no shared cache between users.

Finding 5 — Live Ticker Is Static (Not Live)

The scrolling “RECENTLY LEASED / NEW VACANCY” ticker bar visible at the top of every page is **hardcoded placeholder data**. It does not call the PropertyMax API or any other data source. The entries shown are sample values from the initial build and do not reflect real-time activity.

Finding 6 — Properties Page Has No Timeout Set

The Properties page uses a plain `fetch()` with an `AbortController` but does not set an explicit timeout. It relies on the browser’s default connection timeout (typically 2+ minutes), meaning a slow PropertyMax response could leave the Properties page in a loading state for an extended period with no feedback to the user.

Call Frequency Summary

Scenario	PropertyMax API Calls Generated
1 user opens Home page	3 calls (simultaneous)
1 user opens KPI Dashboard	1 call on load + 1 call every 5 min while open
1 user opens Marketing Reports	1 call on load
1 user opens Properties page	1 call on load
5 users open Home page simultaneously	Up to 15 simultaneous calls
1 user leaves KPI Dashboard open for 1 hour	~13 total calls (1 load + 12 auto-refresh)

Recommendations

Priority	Recommendation	Impact
High	Consolidate the 3 Home page hooks into 1 shared fetch	Reduces Home page calls from 3 → 1 (67% reduction)
High	Activate the server-side <code>managers.getKPIs</code> proxy with a short-lived cache (e.g., 2-minute TTL)	All users share one cached response; removes API key from browser
Medium	Add an explicit timeout to the Properties page fetch	Prevents indefinite loading on slow API responses
Medium	Reduce KPI Dashboard auto-refresh from 5 minutes to on-demand (manual refresh only)	Eliminates background polling for inactive sessions
Low	Wire the Live Ticker to real PropertyMax data	Provides genuine real-time activity visibility